

ThreatForge - Historical ADRs vs Revised Candidate Decisions

NON-CANONICAL CASE-STUDY / WORKING COMPARISON - REVISION 5

Product-semantic evidence baseline: cae0f7b6b37f430ac4e857aabf6ef9f87c89dbb1.

Construction corpus only: MR-0001 and MR-0002. MR-0003, MR-0004 and MR-0005 remain holdouts and do not influence these rewrites.

Purpose

This document compares each historical ThreatForge ADR in the construction corpus with a **single revised candidate rewrite** under the current Decision study. The historical semantic bodies and the revision-4 revised candidate ADR bodies are preserved; revision 5 changes the **model lens**, not the sixteen candidate choices.

The repeated **Status: Draft** section is omitted because lifecycle/governance is outside the present semantic-document study. The revised ADRs are **not ThreatForge patches**: they are experiments used to test whether the emerging Decision model preserves real project knowledge while keeping general DDTA semantics separate from the tool that supports them.

Revision-5 reading rules

1. **Semantic-owner separation** - DDTA/metamodel owns portable semantics; a governed project owns concrete governed instances and project-specific choices; ThreatForge supports the model without silently defining it.
2. **Complete canonical shape** - every field declared by a governed document model is materially present in every canonical representation. A missing field is not the same as an explicitly nullable field whose value is `null`.
3. **No optional-by-omission** - optional semantic value is expressed by field-specific nullability or a canonical empty value, not by silently deleting the field from the representation.
4. **Semantic/representation separation** - meaning, value domains, cardinalities, nullability and invariants are semantic; order, headings, containers, mirrors and serialization mapping are representation-profile concerns.
5. **Single-authority executable projections** - validators, editor schemas/diagnostics, authoring support and LLM guidance should derive deterministically from the governed canonical model instead of copying field inventories or rules.
6. **ADR boundary unchanged** - these representation rules are not inserted as ad-hoc fields or implementation detail inside each Decision. They constrain the later canonical document-model layer.

Reading questions

When comparing each pair, ask:

1. Does the revised Context identify a real **decision-local** problem rather than explain the whole parent MR?
2. Is the Decision precise about the choice without becoming its complete implementation specification?
3. Does the wording preserve the correct semantic owner: DDTA/metamodel, governed project, ThreatForge tool, or ThreatForge project operations?
4. If ThreatForge were replaced by another tool, would a statement that should remain true still be owned by DDTA rather than by the tool?
5. Are persistent/cross-cutting rules preserved as governed intent without adding ad-hoc fields such as **applicability**, **vocabulary** or **patterns**?
6. Does **Consequences** expose the important trade-offs of the choice?
7. Does the redistribution note move only independently checkable obligations/planning downstream, or has it accidentally discarded part of the choice?
8. When a revised ADR mentions a canonical model, contract, projection, validator or editor behavior, does it preserve one semantic authority and let representation/executable detail derive downstream?
9. Could the decision eventually survive a change of programmer/LLM through a future governed effective-context/coverage mechanism rather than human memory?

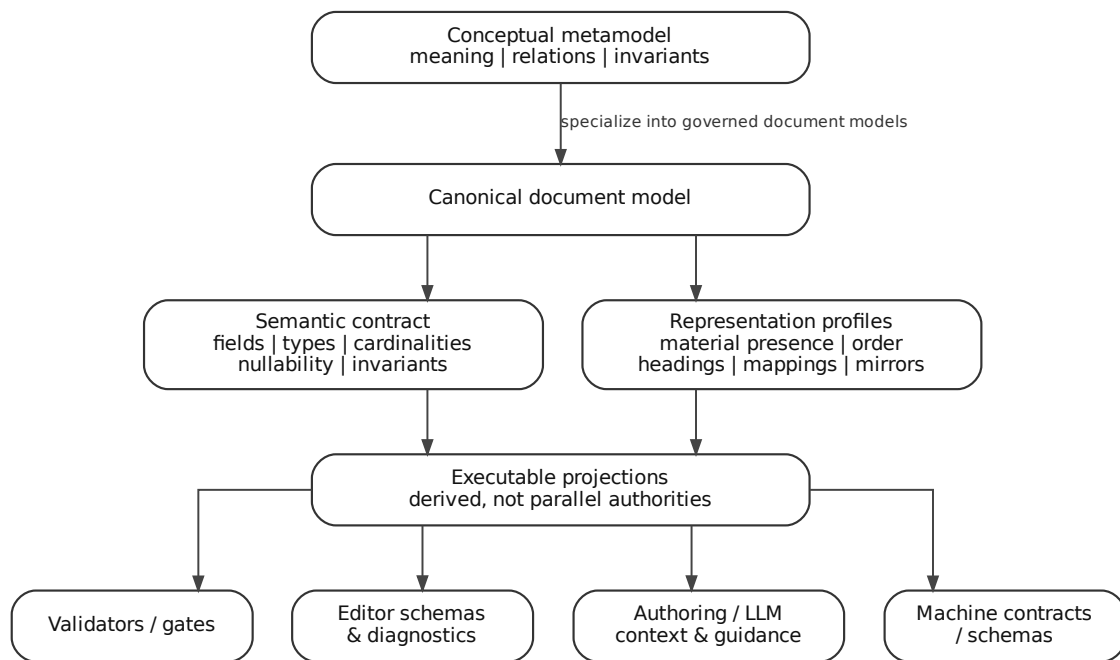


Figure 1: Model to executable projections

Two concrete cross-document evidence cases retained

- **ADR-0002 Vocabulary** exposes a failure mode in the current ThreatForge decomposition: the Decision and canonical vocabulary still exist, but the MR-0001 Requirement registry has no Requirement directly owned by ADR-0002; current vocabulary tooling is instead governed through later ADR-0004 Requirements. Persistent intent can therefore become less explicit even when artifacts survive.
- **ADR-0006 Handoff** is a positive counterexample: it has dedicated downstream Functional/Governance Requirements, including explicit continuity for a new LLM/session. It demonstrates how an operational Decision can remain connected to verifiable downstream obligations.

These cases constrain the **overall documentation model**; they are not reasons to add vocabulary- or handoff-specific fields to **Decision**.

MR-0001 - Gestione documentale governata

MR-0001 / ADR-0001 - Adozione del Diátaxis framework per la classificazione documentale

Historical ADR semantic body

Context La documentazione ThreatForge distingue documenti scritti per apprendere, eseguire un compito, consultare un riferimento o comprendere il modello.

Senza una classificazione documentale esplicita, tutorial, how-to guide, reference ed explanation possono essere confusi. Questa confusione rende incerta l'autorità del contenuto e può portare a usare spiegazioni come se fossero sorgenti normative.

Decision ThreatForge adotta il Diátaxis framework come modello di classificazione documentale.

Le categorie documentali Diátaxis sono:

- tutorial;
- how-to guide;
- reference;
- explanation.

Nel modello ThreatForge:

- i documenti **tutorial** guidano l'apprendimento progressivo;
- i documenti **how-to** guidano azioni operative;
- i documenti **reference** descrivono sorgenti consultabili e includono materiale normativo quando sono collegati a Macro-requirement, Decision, Requirement, registri, grafi o controlli;
- i documenti **explanation** chiariscono il modello per persone e LLM, senza costituire fonte canonica di Requirement, registry, schema, procedure verificabili o controlli deterministici.

Diátaxis classifica la funzione del documento e non sostituisce registri, grafi, Requirement, asset registry, vocabolari controllati o controlli deterministici.

La decisione comprende l'uso di Diátaxis come classificazione documentale, la distinzione tra le quattro categorie e la separazione tra documentazione normativa e documentazione esplicativa. I Requirement derivati applicano questa classificazione ai documenti governati.

Consequences

- Benefit: La documentazione acquisisce una classificazione comprensibile e stabile.
- Benefit: I documenti esplicativi restano distinti dalle sorgenti normative.
- Benefit: I documenti di riferimento rimangono precisi e controllabili.
- Benefit: Programmatori, persone non tecniche, LLM e controlli deterministici distinguono il ruolo previsto di ogni documento.
- Cost: La categoria Diátaxis richiede una fonte canonica unica governata separatamente.
- Risk: Diátaxis può essere applicato in modo errato quando viene trattato come semplice struttura di cartelle anziché come classificazione basata sul bisogno del lettore.
- Cost: La distinzione tra **reference** ed **explanation** richiede revisione editoriale oltre al controllo automatico.
- Constraint: La classificazione Diátaxis non sostituisce le altre fonti canoniche e i controlli governati.

Non-goals

- Definire la fonte canonica della categoria Diátaxis
- Definire la convenzione dei percorsi documentali
- Definire i campi minimi del frontmatter documentale
- Definire il ciclo di vita documentale
- Definire la generazione del libro PDF o delle viste HTML
- Definire tutti i controlli deterministici della documentazione

- Introdurre RDF, SKOS, SHACL o OWL come formato obbligatorio

Revised candidate ADR

Revised title Diátaxis come classificazione comune della funzione documentale

Context Un metodo di documentazione governata applicato a progetti diversi deve permettere a lettori, programmatori, strumenti e LLM di riconoscere rapidamente **per quale bisogno di lettura** esiste un documento. Lasciare a ogni progetto una classificazione proprietaria aumenta il costo di apprendimento e rende meno trasferibile l'esperienza maturata tra progetti.

Occorre quindi scegliere se il metodo documentale adotti una classificazione condivisa e riconoscibile oppure lasci che ogni progetto inventi autonomamente la propria organizzazione funzionale.

Decision Il metodo DDTA adotta **Diátaxis** come classificazione comune della funzione editoriale dei documenti, distinguendo **tutorial**, **how-to guide**, **reference** ed **explanation** secondo il bisogno del lettore.

La classificazione editoriale non determina da sola l'autorità semantica del contenuto: l'autorità continua a dipendere dal modello documentale governato e dalle fonti che possiedono l'informazione.

Consequences

- Benefit: progetti governati diversi possono condividere un'organizzazione leggibile senza una tassonomia proprietaria per progetto.
- Benefit: nuovi lettori e agenti possono riusare conoscenza precedente su come orientarsi nella documentazione.
- Cost: la corretta classificazione richiede giudizio editoriale e non può essere ridotta alla sola posizione del file.
- Risk: Diátaxis può essere degradato a convenzione di cartelle perdendo il legame con il bisogno del lettore.

Redistribution / ownership notes

- **Ownership candidate: DDTA method**, non ThreatForge tool. Se accettata definitivamente, questa scelta deve vivere nella specifica del metodo e ThreatForge deve soltanto supportarla.
- Template, rappresentazione, assistenza di authoring e controlli sono downstream e non definiscono il significato della classificazione.
- Le vecchie liste di Non-goals non vengono ripetute: una falsa interpretazione va corretta in **Context** + **Decision**, non patchata con esclusioni.

MR-0001 / ADR-0002 - Vocabolario controllato della documentazione governata

Historical ADR semantic body

Context La documentazione governata usa termini ricorrenti come Macro-requirement, Decision, Requirement, registro, asset, fonte canonica e output derivato.

Quando questi termini restano liberi, documenti diversi possono usare sinonimi o significati divergenti. Il corpus diventa meno leggibile per persone, sviluppatori e LLM e più difficile da sottoporre a controlli deterministici.

Decision ThreatForge adotta un vocabolario controllato minimo per i termini della documentazione governata.

Il vocabolario controllato costituisce la fonte canonica dei termini documentali centrali. Ogni termine contiene almeno:

- identificativo stabile;
- nome canonico;
- stato;
- definizione sintetica.

Il vocabolario rappresenta anche label controllate con ruolo esplicito, lingua, ragione d'uso, alias ammessi, traduzioni, label candidate e label vietate.

Il registro governato del vocabolario risiede nel percorso logico:

`docs/reference/project-model/registers/vocabularies/documentation-terms.registry.yml`

Il testo normativo privilegia i termini presenti nel vocabolario controllato. Un termine di dominio non ancora registrato entra nel processo come candidato da valutare, anziché come sinonimo libero.

La decisione comprende l'esistenza del vocabolario, il suo ruolo di fonte canonica, il percorso iniziale del registro e la base per future metriche deterministiche di qualità terminologica. Gli incrementi derivati applicano il vocabolario ai documenti governati e separano le decisioni sulle metriche di qualità e sul registro asset.

Consequences

- Benefit: I termini centrali della documentazione hanno una fonte canonica.
- Benefit: I sinonimi liberi diventano rilevabili e discutibili.
- Benefit: Persone, sviluppatori e LLM dispongono di un riferimento esplicito per interpretare il corpus documentale.
- Benefit: I futuri report di qualità possono confrontare i documenti con un registro controllato.
- Cost: Il vocabolario richiede manutenzione e disciplina editoriale.
- Risk: Un vocabolario troppo ampio può diventare rumoroso e poco utile.
- Risk: Un vocabolario troppo piccolo può lasciare ambiguità e sinonimi non governati.
- Constraint: Le label utili alla leggibilità non creano fonti canoniche alternative.

Non-goals

- Definire tutte le tassonomie del progetto
- Definire il registro asset
- Definire formule, soglie o gate di qualità del corpus
- Definire algoritmi di term extraction
- Adottare RDF, SKOS, SHACL o OWL come formato obbligatorio
- Implementare tool o controlli automatici

Revised candidate ADR

Revised title Vocabolario governato come fonte unica del linguaggio condiviso

Context La documentazione governata usa concetti che devono mantenere significato coerente attraverso documenti, progetti, sessioni e agenti differenti. Se definizioni e sinonimi vengono ricostruiti localmente, il corpus può divergere semanticamente anche quando ogni singolo documento resta formalmente valido.

Occorre quindi scegliere se il significato dei termini condivisi dipenda dalla memoria e dalle convenzioni locali oppure sia governato da una fonte comune.

Decision Il metodo DDTA governa i termini condivisi attraverso **una fonte unica di vocabolario controllato**. Le definizioni comuni vengono referenziate o risolte dalla loro fonte governata e non copiate cumulativamente nei documenti discendenti.

Termini specifici di un dominio o di una parte del progetto possono essere aggiunti quando necessari senza ridefinire implicitamente i termini condivisi e senza creare proprietari concorrenti dello stesso significato.

Consequences

- Benefit: persone, strumenti e LLM possono ricostruire il significato dei termini senza dipendere dal contesto ricordato da una sessione precedente.
- Benefit: una modifica terminologica ha un proprietario governato invece di richiedere copie sincronizzate in molti documenti.
- Cost: il metodo deve rendere il vocabolario rilevante effettivamente recuperabile durante authoring e review.

- Risk: una fonte canonica che esiste ma non viene propagata ai consumer può sopravvivere formalmente senza influenzare il lavoro reale.

Redistribution / ownership notes

- **Ownership candidate: DDTA method / cross-document governance**, non ThreatForge tool.
- Schema delle entry, label, traduzioni, path e checker sono scelte downstream.
- Il failure mode ThreatForge resta evidenza: ADR-0002 non ha una catena Requirement diretta nel baseline mentre il vocabolario continua a essere consumato da tooling successivo.
- Il meccanismo per rendere il vocabolario effettivamente disponibile ai discendenti resta **OPEN**: non viene risolto aggiungendo campi duplicati a MR/ADR/REQ.

MR-0001 / ADR-0003 - Tracciabilità degli artefatti implementativi previsti e realizzati

Historical ADR semantic body

Context La documentazione governata può citare tool, report, gate, fixture o altri artefatti implementativi collegati alla soddisfazione dei Requirement.

Quando questi artefatti compaiono soltanto nella prosa di Decision, Requirement o how-to guide, risultano difficili da controllare deterministicamente. Possono essere dimenticati, rimanere incompleti o divergere dal codice realmente scritto.

Il progetto necessita di una fonte controllata che colleghi Requirement, artefatti previsti, artefatti implementati, percorsi, comandi di verifica e stato di completamento.

La tracciabilità copre sia il lavoro già implementato sia il lavoro pianificato ma non ancora realizzato.

Decision ThreatForge usa un implementation trace registry per rappresentare gli artefatti implementativi collegati ai Requirement.

Il registro include artefatti pianificati e artefatti implementati.

Ogni artefatto pianificato contiene almeno:

- identificativo governato;
- tipo di artefatto;
- stato;
- Requirement collegati;
- percorso previsto;
- ragione;
- condizione di completamento.

Ogni artefatto implementato contiene almeno:

- identificativo governato;
- tipo di artefatto;
- stato;
- Requirement collegati;
- percorso implementato;
- eventuale comando di verifica.

Gli artefatti pianificati e non ancora implementati producono un warning deterministico. Gli artefatti dichiarati come implementati sono confrontati deterministicamente con l'esistenza dei Requirement collegati, l'esistenza del percorso implementato, la coerenza tra registro e dichiarazioni di tracciabilità nel codice e l'assenza di riferimenti a Requirement inesistenti.

Il registro non sostituisce i Requirement. Il body del Requirement descrive l'obbligo; l'implementation trace registry descrive gli artefatti previsti o realizzati per soddisfarlo.

La decisione copre tool, report, gate, fixture, artefatti di verifica e altri artefatti implementativi collegati ai Requirement. La sequenza di adozione comprende il Requirement funzionale del registro, il Governance Requirement per il controllo di coerenza, il primo registro minimo e il checker dei warning sugli artefatti pianificati non completati.

Consequences

- Benefit: Le promesse implementative non restano soltanto nella prosa della documentazione governata.
- Benefit: Gli artefatti pianificati diventano visibili tramite warning deterministici.
- Benefit: La coerenza tra Requirement, registro e codice è controllabile senza leggere manualmente tutti i body.
- Benefit: Il registro distingue lavoro pianificato, lavoro implementato e lavoro da completare.
- Benefit: Le future relazioni di grafo possono derivare da Requirement, implementation trace registry e dichiarazioni nel codice.
- Cost: Il registro richiede sincronizzazione continua con Requirement e codice.
- Risk: Un registro non aggiornato può introdurre falsi warning o falsa sicurezza.
- Cost: La disciplina iniziale aumenta la verbosità prima dell'automazione completa.
- Risk: Warning non calibrati sugli artefatti pianificati possono diventare rumore ignorato.

Non-goals

- Definire il formato definitivo dello schema del registro
- Definire tutti i valori controllati ammessi
- Implementare il tool di validazione nella decisione stessa
- Definire soglie di qualità del corpus documentale
- Definire il modello completo di generazione del grafo

Revised candidate ADR

Revised title Supporto ThreatForge alla tracciabilità della realizzazione governata

Context I progetti governati possono aver bisogno di rendere osservabile come un obbligo documentato viene pianificato, realizzato e verificato attraverso artefatti differenti. ThreatForge deve poter supportare questa conoscenza senza trasformare la propria implementazione nella fonte del significato dell'obbligo o della relazione prevista dal modello.

Occorre quindi decidere come il tool supporti la tracciabilità della realizzazione mantenendo separata la semantica governata del progetto.

Decision ThreatForge supporta una **tracciabilità implementativa governata** consumando le identità, le relazioni e le regole esposte dal modello del progetto e collegandole alle evidenze o agli artefatti di realizzazione gestiti dal tool.

ThreatForge non rende vero un obbligo per il solo fatto di registrare un artefatto e non definisce autonomamente che cosa significhi soddisfare l'obbligo: la semantica resta posseduta dal modello governato, mentre il tool rende la relazione osservabile, interrogabile e verificabile secondo quel contratto.

Consequences

- Benefit: la conoscenza sulla realizzazione può essere automatizzata senza spostare l'autorità semantica nel tool.
- Benefit: il tool può distinguere pianificazione, realizzazione ed evidenza quando tali distinzioni sono previste dal modello.
- Cost: ThreatForge deve restare allineato alle relazioni e ai contratti del modello che supporta.
- Risk: se il tool introduce stati o significati non previsti dal modello, diventa una fonte semantica concorrente.

Redistribution / ownership notes

- **Ownership split detected.** La necessita e la semantica generale della tracciabilita sono questioni del DDTA/metamodello; questa ADR puo legittimamente decidere solo **come ThreatForge le supporta**.
- Il termine di classe **Requirement** viene evitato nella candidata finche il relativo modello non e chiuso.
- Registry, stati, warning, checker, campi e cardinalita sono downstream/tool design.

MR-0001 / ADR-0004 - Modello di label controllate e valori tassonomici documentali

Historical ADR semantic body

Context Il vocabolario controllato introdotto da ADR-0002 richiede una distinzione più precisa tra termine canonico, alias ammesso, traduzione, label storica, label candidata e label vietata.

Un campo generico come `allowed_labels` è troppo permissivo: può mescolare sinonimi, acronimi, traduzioni, nomi storici, varianti operative e label di visualizzazione. Questa ambiguità rende difficile stabilire quale forma sia canonica, quale sia soltanto leggibile, quale sia accettata per compatibilità e quale sia segnalata dai controlli.

La documentazione governata usa anche campi ricorrenti come `status`, `artifact_type`, `requirement_type`, `decision_type`, `check_status` e ruoli delle label. Stringhe libere in questi campi consentono a registri diversi di usare valori simili con significati differenti.

Il caso più rischioso è `status`: valori come `active`, `draft`, `implemented` o `deprecated` non hanno un significato universale. Il significato cambia in base al registro e al tipo di record. Un `check active` viene eseguito dall'orchestratore; un termine `active` è utilizzabile nel vocabolario; una Decision `accepted` è applicabile; un artefatto `implemented` esiste ed è verificabile. Una tassonomia globale di `status` risulterebbe ambigua.

I nomi temporanei usati per organizzare il lavoro non appartengono ai concetti canonici del modello documentale. Restano utilizzabili nei path tecnici o nelle procedure operative senza diventare termini di dominio nella documentazione governata.

Decision Il vocabolario controllato rappresenta le label tramite un modello esplicito basato su ruolo, lingua e ragione d'uso.

Ogni termine governato mantiene un solo `canonical_name` e una `canonical_language`. Ogni label associata a un termine dichiara almeno:

- `value`;
- `language`;
- `role`;
- `reason`.

I ruoli iniziali delle label sono:

- `preferred`: forma preferita nella documentazione governata;
- `accepted_alias`: alias ammesso per una ragione esplicita, come un acronimo tecnico o una compatibilità storica;
- `translation`: traduzione leggibile e non canonica;
- `forbidden`: forma vietata e segnalabile;
- `candidate`: forma proposta e non ancora accettata;
- `historical`: forma storica riconoscibile e non preferita per nuovo testo.

Un sinonimo non costituisce automaticamente una label ammessa. Ogni alias accettato conserva una ragione esplicita. Le traduzioni supportano la lettura senza creare una seconda fonte canonica. Le label vietate e le frasi temporanee da evitare sono registrate esplicitamente per consentire segnalazioni deterministiche.

I campi documentali e di registro con valori ripetuti evolvono verso value set tassonomici contestuali. Ogni value set dichiara almeno:

- `name`;
- `field_name`;
- `applies_to_registry`;

- `applies_to_record`;
- `status` del value set;
- `description`;
- lista dei valori ammessi con `value` e `meaning`.

Non esiste una tassonomia globale generica per `status` quando lo stesso valore assume significati differenti in registri diversi. I valori controllati sono specifici del contesto, tra cui `check_status`, `implementation_artifact_status`, `decision_status`, `requirement_lifecycle_status`, `vocabulary_term_status` e `field_value_set_status`.

Lo stato di un Requirement resta distinto dallo stato di implementazione. Il lifecycle di un Requirement comprende `draft`, `accepted`, `superseded`, `deprecated` e `removed`; lo stato implementativo deriva dalla tracciabilità e dalle verifiche anziché dal medesimo campo `status`.

Il campo `requirement_type` contiene esclusivamente tipi concreti registrati nel proprio value set contestuale. `specialized` rappresenta una categoria astratta e non compare come valore del campo né come alias di un tipo concreto.

Ogni tipo concreto dichiara, oltre a `value` e `meaning`, l'eventuale appartenenza alla categoria specializzata, la presenza di un Requirement padre e i tipi concreti ammessi come padre. I tipi concreti disponibili sono esclusivamente quelli registrati nel value set contestuale `requirement_type` e associati alle rispettive varianti e regole canoniche. Ogni nuovo tipo entra attraverso un'estensione governata esplicita e non attraverso una lista locale o un alias implicito.

La decisione comprende la sostituzione di `allowed_labels`, la distinzione dei ruoli delle label, i value set contestuali, la scomposizione di `status` e l'esclusione dei nomi temporanei dai concetti canonici. Gli incrementi derivati aggiornano il registro del vocabolario, definiscono i primi value set, introducono il Governance Requirement di coerenza e aggiungono il controllo sulle label e sui valori fuori contesto.

Consequences

- Benefit: Il vocabolario riduce l'ambiguità tra forme canoniche, sinonimi, traduzioni e acronimi.
- Benefit: Le label ammesse conservano una ragione esplicita.
- Benefit: Le traduzioni migliorano la leggibilità senza diventare fonti canoniche.
- Benefit: I controlli futuri distinguono errori, warning e candidati.
- Benefit: I valori dei campi ricorrenti diventano verificabili nel proprio contesto.
- Benefit: Il campo `status` non viene trattato come una lista globale ambigua.
- Benefit: Ogni valore controllato conserva un significato adatto al registro e al record di applicazione.
- Cost: Il registro del vocabolario diventa più verboso.
- Cost: Il registro dei valori tassonomici richiede più record.
- Cost: La classificazione delle label richiede disciplina editoriale.
- Cost: I tool di validazione gestiscono più campi e più contesti.
- Risk: Una label utile alla lettura può sembrare canonica quando il ruolo non è classificato correttamente.

Non-goals

- Definire tutte le tassonomie del progetto
- Implementare il controllo terminologico completo sul corpus nella decisione stessa
- Definire metriche di qualità del corpus
- Definire il registro asset
- Decidere il modello completo di implementation state derivato

Revised candidate ADR

Revised title Risoluzione contestuale dei valori controllati da fonti governate

Context ThreatForge consuma campi e label controllati definiti dalla documentazione governata. Se il tool codifica enum globali o interpreta autonomamente stringhe come `status`, può attribuire significati diversi da quelli previsti dal modello o dal contesto documentale.

Occorre quindi decidere se i consumer ThreatForge mantengano inventari locali oppure risolvano significati e valori dalle fonti governate che li possiedono.

Decision ThreatForge risolve **label e valori controllati nel loro contesto governato** e deriva gli inventari dai cataloghi o contratti del modello/progetto anziché mantenere enum semantici concorrenti nel codice dei consumer.

Il tool può validare che un valore sia ammesso nel contesto dichiarato, ma non introduce autonomamente una semantica universale di **status**, lifecycle o tipo documentale.

Consequences

- Benefit: consumer differenti possono usare gli stessi significati senza duplicare liste locali.
- Benefit: l'evoluzione del modello non richiede di reinterpretare manualmente gli stessi valori in ogni adapter.
- Cost: resolution e provenance delle fonti controllate diventano dipendenze esplicite del tool.
- Risk: cache o proiezioni obsolete possono applicare un significato non più corrente.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge tool support.** La semantica dei value set e dei lifecycle appartiene al modello/metodo che li definirà.
- L'ADR storica contiene anche decisioni generali su label, lifecycle e tipi di requisito che non devono essere fissate indirettamente dal tool.
- Schema concreto, ruoli delle label e tassonomie restano fuori dal Decision metamodel corrente.

MR-0001 / ADR-0006 - Handoff locale come artefatto governato di continuità operativa

Historical ADR semantic body

Context Il lavoro governato può estendersi oltre una singola sessione operativa. In questi casi un handoff riproducibile consente di riprendere il lavoro senza dipendere dalla memoria della chat o da archivi prodotti esternamente.

Lo ZIP di handoff nasce dal repository locale, che costituisce la fonte canonica per stato, registri, check, comandi e contenuto del progetto.

Un handoff soltanto riassuntivo non offre sufficiente continuità per la ripresa dello sviluppo da parte di un LLM: il modello necessita anche dei file governati, del codice, dei contratti, dei test e degli altri sorgenti tracciati.

Decision ThreatForge introduce un handoff archive locale come artefatto governato di continuità operativa.

Un tool locale tracciato nell'implementation trace registry produce l'archive. Il contenuto usa la label canonica di prodotto **ThreatForge** e include esclusivamente il progetto operativo corrente; gli artefatti legacy archiviati sotto **old/** restano esclusi dallo snapshot di continuità.

L'archive include uno snapshot del progetto basato sui file tracciati da Git, così che un LLM o un operatore possa leggere lo stato sorgente necessario per riprendere lo sviluppo. Lo snapshot esclude **.git**, dipendenze installate, cache, build output e artifact generati non tracciati.

Consequences

- Benefit: L'handoff deriva dalla fonte canonica locale anziché dalla sandbox della chat.
- Benefit: Lo stato del repository e l'output dei check vengono raccolti dal tool locale.
- Benefit: L'archive supporta la continuazione in una nuova chat o la consegna del contesto a un operatore.
- Cost: L'archive aumenta di dimensione perché contiene lo snapshot dei file tracciati.
- Constraint: Il tool resta sotto MR-0001 perché impacchetta documentazione governata, registri, sorgenti tracciati e verifiche di continuità.
- Constraint: Lo snapshot include soltanto il progetto operativo corrente e gli artefatti Git tracciati ammessi.

Revised candidate ADR

Revised title Handoff governato per la continuità dello sviluppo ThreatForge

Context Lo sviluppo di ThreatForge attraversa sessioni di lavoro, programmatori e LLM differenti. Affidare la continuità alla memoria di una chat o a un riassunto manuale rende facile perdere Decision, stato del repository, verifiche e sorgenti necessarie a proseguire il lavoro in modo riproducibile.

Occorre quindi decidere se la continuità operativa dello sviluppo ThreatForge resti una pratica informale oppure venga resa ricostruibile a partire dallo stato governato del progetto.

Decision Il processo di sviluppo di ThreatForge usa un **handoff governato e ricostruibile** derivato dallo stato canonico del repository e dalle evidenze operative necessarie a riprendere il lavoro.

L'handoff non sostituisce le fonti governate e non rende canonico il riassunto della sessione: serve a trasportare abbastanza contesto e sorgenti per consentire al nuovo agente di verificare e ricostruire lo stato rilevante.

Consequences

- Benefit: il cambio di sessione o agente non dipende dalla memoria dell'operatore precedente.
- Benefit: la continuità viene verificata rispetto allo stato del progetto, non rispetto a una narrazione isolata.
- Cost: l'handoff deve essere mantenuto riproducibile e sufficientemente completo.
- Risk: un handoff incompleto o obsoleto può dare falsa confidenza sulla continuità.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge project operations**, non regola universale DDTA e non semantica del tool verso i target project.
- Il caso resta importante per la tesi come evidenza pratica di **recoverability** e perdita di contesto tra agenti.
- ZIP, manifest, snapshot, dry-run e filtri sono downstream operational requirements/implementation.

MR-0001 / ADR-0007 - Canonical document models and model-oriented validation

Historical ADR semantic body

Context ThreatForge governs Macro-requirements, decisions and requirements through YAML registry records and Markdown bodies. Existing checks validate selected fragments of those representations, but the structural rules are distributed across independent tools and do not yet form an explicit document model.

The governed corpus also contains inconsistent headings, languages, punctuation and normative forms. Live authoring assistance, deterministic migration and reliable LLM analysis require one canonical description of each document model and stable diagnostics shared by gates and editors.

Decision ThreatForge defines governed document models through the canonical document model index. Each active model entry references exactly one YAML registry profile and exactly one Markdown body profile, and the profiles referenced by active model entries form the active representation-profile inventory. This Decision does not impose a fixed number of models or profiles.

Canonical model definitions are organized by model and representation. A shared validation engine consumes those definitions, while one endpoint validates each complete logical model and one cross-model endpoint validates relationships among models.

Model checkers absorb overlapping partial document checks after equivalent rule coverage is proven. Every diagnostic uses a stable rule identifier that is shared by repository gates, migration reports, editor diagnostics, quick fixes and deterministic fixtures.

Canonical identifiers, YAML field names, Markdown headings and persisted controlled values are English. Future translations are presentation metadata and do not create alternative canonical identifiers or values.

New model checks are introduced as non-blocking planned checks. They become active only after every active registry record and referenced body has been migrated and the complete model report contains no errors. ThreatForge does not retain a legacy validation mode for the previous body formats.

Consequences

- Benefit: The complete rule set for one document type is discoverable from one logical model.
- Benefit: Repository gates, VS Code and the future web editor can use identical rule identifiers and semantics.
- Cost: Existing governed records and bodies require a reviewed repository-wide migration.
- Risk: Activating model checks before migration would make the governed repository gate fail.
- Constraint: Partial checks may be removed only after the replacement model checker proves equivalent or stronger coverage.

Non-goals

- Immediate implementation of the validation engine
- Immediate activation of new blocking checks
- Governance of ordinary Diátaxis documentation as governed document models

Revised candidate ADR

Revised title Contratti eseguibili dei modelli documentali consumati da ThreatForge

Context Validatori, editor, generatori e migrazioni ThreatForge devono interpretare gli stessi modelli documentali. Se ogni consumer ricostruisce autonomamente struttura, campi e regole, il tool può produrre comportamenti divergenti e finire per ridefinire implicitamente il modello che dovrebbe soltanto supportare.

Occorre quindi decidere come ThreatForge consumi in modo uniforme la descrizione governata dei modelli documentali.

Decision ThreatForge usa **contratti eseguibili o proiezioni canoniche dei modelli documentali governati** come fonte comune per validazione, authoring, migrazione e altri consumer. I consumer derivano le proprie regole da tali contratti anziché mantenere descrizioni semantiche locali concorrenti.

Il contratto eseguibile è una rappresentazione consumabile dal tool: non rende ThreatForge proprietario della semantica generale del documento, che resta definita dal modello DDTA e dalle estensioni governate del progetto.

Consequences

- Benefit: consumer differenti possono condividere interpretazione e diagnostica senza duplicare il modello nel codice.
- Benefit: l'evoluzione del modello può essere propagata attraverso una fonte consumabile comune.
- Cost: i contratti devono restare tracciabili alla fonte semantica che rappresentano.
- Risk: una proiezione obsoleta o arricchita con semantica tool-specifica può diventare una fonte concorrente.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge tool architecture.** Il tool possiede il modo in cui consuma il modello, non il significato generale dei tipi documentali.
- Cardinalità dei profili, lingua dei campi, activation sequence e migrazione sono downstream.
- Il rapporto esatto tra metamodello semantico e contratto eseguibile resta da definire nella futura mapping/serialization phase.

MR-0001 / ADR-0008 - Canonical governed entity references in Markdown bodies

Historical ADR semantic body

Context Governed Markdown bodies currently represent document identity in their headers and controlled values in profile-defined sections, but they lack one canonical representation for references to other governed

entities.

Base Analysis Elements introduce the first immediate use case. The same problem also applies to future threats, findings, mitigations, snapshots and other canonical records. Free-form alternatives such as bare identifiers, parenthesized tokens, Markdown links or independently authored labels create ambiguous parsing and allow readable titles to diverge from their authoritative records.

A reference also has different semantics from an origin relation. Text containing an identifier does not establish when or why the referenced entity came into existence.

Decision ThreatForge adopts one canonical governed entity reference payload:

```
[<canonical-id>] <canonical-title>
```

The canonical identifier is authoritative. The canonical title is a required human-readable mirror resolved from the entity's authoritative governed source. Exactly one ASCII space separates the closing bracket from the title, and the payload contains no surrounding whitespace.

The containing body profile defines the reference-bearing position and any Markdown container outside the payload. A list marker, section heading or other profile-owned structure is not part of the reference payload. Identical text outside a declared reference-bearing position remains ordinary Markdown prose.

Each referenceable governed entity type is associated with one registered canonical resolver. The resolver identifies the authoritative source, canonical title, entity type and semantic eligibility rules for the current document and reference position.

A canonical reference resolves to exactly one existing entity of an allowed type. Unknown identifiers, ambiguous identifiers, disallowed entity types, divergent titles and failed eligibility rules are invalid reference states.

Alternative forms such as (@BAE-0001) Title, a bare BAE-0001, or [BAE-0001](target) are not canonical governed entity references and receive no compatibility alias.

A textual reference does not create, originate or justify the referenced entity. Origin, provenance, lifecycle and reference eligibility remain owned by the entity's governing model. For BAE records, MR-0003 defines those semantics, including the prohibition against references to an element introduced only by a descendant document.

MR-0001 owns the shared reference representation and generic resolution contract. Entity-owning Macro-requirements own entity semantics and eligibility. MR-0002 authoring and editor adapters consume both sources without duplicating either rule set.

The exact grammar contract, profile declarations, resolver registration, diagnostics and verification behavior are specified by follow-up Requirements.

Consequences

- Benefit: Humans, deterministic tools and LLMs receive one readable and machine-resolvable reference form.
- Benefit: Future governed entity types reuse the same representation without BAE-specific syntax duplication.
- Benefit: Canonical titles remain visible while identifiers preserve stable identity.
- Cost: Reference-bearing body profiles require explicit position and container declarations.
- Cost: Every referenceable entity type requires an authoritative resolver and source projection.
- Risk: Treating identifier-like prose as a governed reference could create false diagnostics.
- Risk: Stale title mirrors could mislead readers until corrected.
- Constraint: Only profile-declared reference-bearing positions receive governed reference semantics.
- Constraint: The identifier remains authoritative and the title remains a derived mirror.
- Constraint: Reference resolution remains side-effect-free.
- Constraint: Compatibility aliases do not expand the canonical grammar.

Non-goals

- Define BAE origin, provenance, topology or lifecycle semantics
- Define editor completion ranking or user-interface presentation
- Convert every identifier occurrence in existing prose into a governed reference

- Define external URL or citation syntax
- Create concrete BAE, threat, finding or mitigation records
- Introduce compatibility aliases for alternative reference forms

Revised candidate ADR

Revised title Rappresentazione Doc-as-Code canonica dei riferimenti governati

Context Quando il modello governato permette a un documento di riferirsi a un'altra entita, la rappresentazione Markdown deve poter distinguere un riferimento governato da testo ordinario e deve preservare identita stabile e leggibilita senza ridefinire la semantica dell'entita referenziata.

Occorre quindi scegliere una rappresentazione Doc-as-Code comune per serializzare tali riferimenti.

Decision La rappresentazione Doc-as-Code studiata per DDTA usa un payload di riferimento canonico nella forma [`<canonical-id>`] `<canonical-title>`, in cui l'identificativo rappresenta l'identita e il titolo e un mirror leggibile risolto dalla fonte governata.

La rappresentazione di riferimento non crea origine, provenienza, derivazione o altra relazione semantica: tali relazioni restano definite dal modello dell'entita e dal contesto che ammette il riferimento.

Consequences

- Benefit: persone e tool possono leggere e risolvere la stessa rappresentazione senza affidarsi a forme libere.
- Benefit: la serializzazione resta separata dalla semantica delle relazioni piu forti.
- Cost: parser e authoring devono conoscere le posizioni in cui il payload ha significato governato.
- Risk: confondere la presenza dell'ID con una relazione semantica introdurrebbe inferenze non autorizzate.

Redistribution / ownership notes

- **Ownership candidate: DDTA representation/serialization**, non ThreatForge tool e non Decision semantic core.
- La scelta e mantenuta come candidata storica, ma la sua chiusura definitiva e **deferred** finche identita, reference semantics e mapping Doc-as-Code non sono stabiliti.
- Resolver, whitespace, diagnostica e compatibility aliases sono dettagli downstream.

MR-0001 / ADR-0009 - Governed Security Requirement specialization and finding derivation

Historical ADR semantic body

Context ThreatForge governs its document corpus through an extensible canonical model catalog. The common analysis model establishes Common Findings as methodology-neutral and traceable results originating from methodology-specific Analysis Records, but the governed corpus has no document model capable of representing the security obligations introduced to address accepted Findings.

Treating those obligations as Governance Requirements would confuse required product security behavior with repository governance, validation and verification controls. Treating them as ordinary independent Functional Requirements would lose their child relationship with the function they protect. Binding them directly to methodology plugins or method-specific classifications would instead couple product obligations to one analytical vocabulary and would obscure the common Finding boundary between analysis and governed product requirements.

The project also needs a complete documentary trace from the methodology-specific analysis to the resulting security obligation without introducing a second canonical inventory for Common Findings. Common Finding source files already own stable identity, originating Analysis Record, affected subjects, analytical content and current review state. Security Requirement documents can therefore preserve the downstream derivation relation while leaving Finding identity and methodology-specific evidence with their existing owning models.

Decision ThreatForge introduces Security Requirement as a logical governed document model extension. A Security Requirement expresses an independently testable security obligation and is a child specialization of exactly one governed Functional Requirement. It remains owned by the same Macro-requirement and Decision as its Functional Requirement parent because the security obligation belongs to the governed definition of the protected product behavior.

A Security Requirement is authored only after one or more Common Findings have been produced and explicitly reviewed as accepted. The author selects the accepted Common Findings used to justify the obligation and declares each selected Finding exactly once in the governed Finding derivation section of the Security Requirement Markdown body. Each referenced Common Finding includes the Security Requirement parent Functional Requirement among its affected governed Functional Requirements. A Finding can affect multiple Functional Requirements and can contribute to distinct Security Requirements under each affected parent. A Security Requirement can also consolidate one obligation supported by multiple accepted Findings. These relationships do not impose one-to-one cardinality and no automatic discovery, consolidation or requirement generation is implied.

The Security Requirement references Common Findings rather than methodology plugins, method identifiers or method-specific classifications. Each Common Finding preserves its originating Analysis Record, while the Analysis Record and the method-specific analytical documentation preserve the method, payload, classifications, applicability reasoning and methodological evidence. The complete documentary provenance is therefore navigable from Security Requirement to Common Finding to Analysis Record and to the methodology-specific analytical material without copying method semantics into the product requirement.

Security Requirement records share the governed requirement registry and use a distinct record variant discriminated by `requirement_type` security. The registry record preserves structural identity and parentage through `id`, `title`, `status`, `requirement_type`, `macro_requirement_id`, `decision_id`, `parent_requirement_id` and `body_path`. Finding references are not duplicated in the registry. They are owned by the dedicated Markdown body profile, whose canonical sections are Intent, Parent Functional Requirement, Finding derivation, Security obligation, Scope and Acceptance. The parent reference in the body is a readable governed mirror of `parent_requirement_id`. The Finding derivation references are the canonical downstream relation from the Security Requirement to the selected Common Findings.

The authoritative Common Finding inventory remains the repository-contained set of validated `.analysis-finding.yml` files. ThreatForge does not introduce a central Common Finding registry for Security Requirement derivation. Consumers can derive a consultable projection that combines Finding identity, current review state, originating Analysis Record, affected subjects, source path and referring Security Requirements by reading validated Finding sources and governed Security Requirement references. Such a projection is derived information and is not a second canonical source.

Security Requirement validation resolves the Functional Requirement parent and Common Finding references through their owning canonical models. It checks the current accepted state and parent-to-Finding coherence, but it does not interpret method-specific payloads, rediscover methodology classifications, execute a plugin or require the originating plugin to be available. The complete operational lifecycle of Common Finding review transitions and the workflow for revising existing Security Requirements after later state changes remain outside this Decision.

A Security Requirement is distinct from a Governance Requirement. The former constrains product behavior for security, while the latter defines deterministic governance, validation or verification obligations. Whether Governance Requirements can later govern Security Requirements remains a separate activation step and is not required to introduce the Security Requirement model itself.

Consequences

- Benefit: Security obligations remain child requirements of the functional behavior they protect.
- Benefit: Product obligations remain independent from the vocabulary and availability of a particular methodology plugin.
- Benefit: Method-specific evidence remains available through the Common Finding and originating Analysis Record provenance chain.
- Benefit: Findings from different methods can support independent or consolidated Security Requirements without being merged automatically.

- Benefit: A complete consultable trace can be derived without creating a duplicate Common Finding registry.
- Benefit: Governance Requirements retain a meaning distinct from product security behavior.
- Cost: The active canonical document model catalog must add and govern the Security Requirement model.
- Cost: The requirement registry body profiles authoring consumers and cross-model validation require coordinated extension.
- Cost: Authors must select and explain the accepted Findings used to derive each Security Requirement.
- Risk: Poorly scoped Security Requirements could duplicate their Functional Requirement parent.
- Risk: Weak Finding derivation prose could make an otherwise valid relation difficult for reviewers to understand.
- Risk: Excessive consolidation could hide independently actionable Findings.
- Constraint: Every Security Requirement has exactly one Functional Requirement parent.
- Constraint: Every Security Requirement remains in the same Macro-requirement and Decision as its parent.
- Constraint: Every Security Requirement declares at least one accepted Common Finding in its body.
- Constraint: Every referenced Common Finding includes the parent Functional Requirement among its affected governed Functional Requirements.
- Constraint: Each Common Finding selected for the Security Requirement derivation appears exactly once in the Finding derivation section.
- Constraint: Security Requirements do not contain method-specific classifications payloads applicability rules failure modes or attack classes.
- Constraint: Security Requirement creation and reference resolution never modify the originating Analysis Record or Common Finding.
- Constraint: Repository-contained validated .analysis-finding.yml files remain the canonical Common Finding sources.

Non-goals

- Define STRIDE categories or analysis rules
- Define STRIDE-AI categories failure modes or attack classes
- Generalize Common Findings to performance reliability privacy or other non-security analysis domains
- Support Security Requirements justified only by policy standards compliance baselines or contracts
- Define quantitative risk scoring
- Define a security control catalogue
- Automatically discover consolidate generate or accept Security Requirements
- Introduce a central Common Finding registry
- Define the complete Common Finding review workflow transition authorization or remediation lifecycle
- Require Security Requirement validation to execute or interpret methodology plugins
- Activate Governance Requirement children of Security Requirements
- Implement the Security Requirement model in this Decision

Revised candidate ADR

Revised title Supporto ThreatForge a obblighi di sicurezza governati derivati dall'analisi

Context Se il modello DDTA distingue obblighi di sicurezza collegati a risultati di analisi revisionati, ThreatForge deve poterli creare, risolvere e verificare senza incorporare nel tool la semantica della specializzazione, le cardinalità del modello o il vocabolario di una metodologia di analisi specifica.

Occorre quindi decidere come il tool supporti tale eventuale estensione del modello mantenendo separati semantica governata e meccanismi di authoring/validation.

Decision ThreatForge supporta le **specializzazioni di obblighi di sicurezza e le relative relazioni di derivazione** esclusivamente attraverso i contratti governati del modello che le definiscono. Il tool risolve parentage, riferimenti e coerenza secondo tali contratti e conserva la catena verso i risultati di analisi senza copiare nel documento di prodotto semantica specifica del metodo analitico.

ThreatForge non decide autonomamente la semantica della specializzazione di sicurezza, le sue cardinalità o gli stati che rendono utilizzabile un risultato analitico: tali scelte appartengono al metamodello e ai documenti di

analisi che le possiedono.

Consequences

- Benefit: il tool può supportare estensioni di sicurezza senza trasformare implementazioni correnti in definizioni del metamodello.
- Benefit: la separazione fra risultato di analisi e obbligo di prodotto può essere preservata anche con metodologie differenti.
- Cost: authoring e validation dipendono da contratti cross-model espliciti.
- Risk: hard-code di parentage, stati o categorie nel tool creerebbe una seconda versione del modello.

Redistribution / ownership notes

- **Strong historical ownership mismatch.** L'ADR originale definisce soprattutto semantica del futuro Requirement/analysis metamodel; la candidata ThreatForge può solo definirne il supporto.
- Parent FR, cardinalità, Finding acceptance, body sections e lifecycle devono essere rivalutati nelle rispettive fasi del metamodello e non sono chiusi da questa ADR.
- Questo caso diventa un test esplicito per impedire che ThreatForge anticipi il modello che sta aiutando a studiare.

MR-0001 / ADR-0010 - Registry-derived extensibility of governed document models

Historical ADR semantic body

Context ADR-0007 established canonical model-oriented validation for governed documents. Some shared loaders and consumers nevertheless encoded the active model inventory through fixed cardinalities or locally authored lists. ADR-0009 requires Security Requirement to enter the catalog as an additional governed document model, exposing the need for an explicit extension boundary that preserves one canonical inventory and prevents consumer-specific inventories.

Decision ThreatForge treats the canonical document model index as the inventory of active governed document models and the referenced representation profiles as the corresponding profile inventory. Shared loaders, validators, authoring catalogs, assistance engines and other consumers derive their active inventories from those canonical sources or from deterministic projections. Numeric model and profile counts are observations of a repository state and never define the lasting shape of shared loading, validation or consumer infrastructure. A model extension consists of a canonical model entry, resolvable representation profiles, stable identifiers, controlled-value references, validation coverage and declared consumer coverage. Filesystem discovery and consumer-local model lists have no canonical role. ADR-0007 owns canonical model-oriented validation, while ADR-0009 owns Security Requirement semantics and its governed extension of the catalog.

Consequences

- Benefit: Security Requirement can be introduced as a governed model extension instead of a shared-loader exception.
- Benefit: Future document models reuse one explicit catalog and deterministic extension boundary.
- Benefit: Consumers remain aligned with the same canonical model and profile inventory.
- Cost: Existing consumers with consumer-local model inventories or closed dispatch require governed refactoring.
- Cost: Model activation requires explicit source validation and consumer coverage.
- Risk: A model could be registered before every required consumer supports it.
- Risk: Overloading the model descriptor could transfer consumer-specific behavior into the canonical source.
- Constraint: Model and profile discovery remains explicit and registry-derived.
- Constraint: Numeric model and profile cardinalities are not canonical architecture constraints.
- Constraint: Model-specific semantics remain owned by their governing Decisions and Requirements.

Non-goals

- Implement the Security Requirement model
- Modify the current model or representation profile registries
- Implement model-specific authoring or Markdown assistance providers in this Decision
- Enable Governance Requirement children of Security Requirements
- Reintroduce the legacy graph or append-first mechanisms

Revised candidate ADR

Revised title Consumer ThreatForge estensibili dall'inventario governato dei modelli

Context Quando il modello governato evolve con nuovi tipi documentali, consumer ThreatForge che mantengono liste o cardinalità hard-coded possono continuare a funzionare secondo un inventario diverso da quello realmente attivo.

Occorre quindi decidere se l'estensione del modello richieda patch specifiche in ogni consumer oppure se l'inventario supportato sia derivato da una fonte governata comune.

Decision ThreatForge deriva **l'inventario dei modelli documentali supportati e dei relativi contratti** dal catalogo governato o dalle sue proiezioni deterministiche, evitando inventari locali e cardinalità fisse nei consumer generici.

I consumer possono avere provider specifici quando necessario, ma la presenza e l'identità dei modelli non vengono ridefinite localmente dal tool.

Consequences

- Benefit: nuovi modelli possono entrare nel tool attraverso un'estensione esplicita invece di una serie di eccezioni scollegate.
- Benefit: i consumer possono essere verificati contro lo stesso inventario attivo.
- Cost: l'attivazione richiede coverage dichiarata dei consumer che devono supportare il nuovo modello.
- Risk: registrare un modello prima che i consumer necessari lo supportino può produrre una falsa estensibilità.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge tool architecture.** Il catalogo descrive modelli posseduti dal DDTA/progetto; ThreatForge decide come derivarne il proprio dispatch.
- La semantica dell'estensione e dei singoli modelli non appartiene a questa ADR.

MR-0002 / ADR-0001 - Separazione tra core ThreatForge, app e adapter VS Code

Historical ADR semantic body

Context Il catalogo di task VS Code governato da MR-0002/ADR-0005 ha dimostrato che i tool locali possono essere esposti dall'editor senza duplicarne la logica.

I task attuali coprono check e simulazioni, ma non definiscono ancora l'architettura per guidare la produzione di artefatti implementativi.

La scelta dell'ambiente di authoring evita che regole, identificativi, path e tracciabilità dipendano da una singola interfaccia.

Decision ThreatForge separa tre livelli:

- il core e i tool governati contengono contratti, validazioni, generazione e tracciabilità canonica;
- l'app ThreatForge orchestra progetti, Requirement, workflow e risultati;
- Visual Studio Code costituisce il primo IDE adapter per l'authoring tecnico.

L'adapter VS Code fornisce task, comandi, selezioni guidate e viste dedicate invocando capacità governate senza reimplementarle.

Visual Studio e altri IDE restano adapter futuri. La loro introduzione conserva inalterate le regole canoniche del core.

Il primo incremento pianifica un artefatto implementativo in modalità read-only a partire da un Requirement governato esistente.

Consequences

- Benefit: VS Code offre la superficie iniziale di authoring senza diventare fonte canonica delle regole.
- Benefit: L'app ThreatForge rimane il centro futuro di orchestrazione del prodotto.
- Benefit: I tool CLI restano riutilizzabili da app, editor, CI e LLM.
- Cost: Una custom extension VS Code entra nel prodotto quando task e comandi semplici non risultano più sufficienti.
- Constraint: Il supporto Visual Studio entra in seguito come adapter separato.

Revised candidate ADR

Revised title Separazione tra capacità ThreatForge, orchestrazione applicativa e adapter

Context ThreatForge deve esporre le stesse capacità di supporto alla documentazione governata attraverso CLI, editor e applicazione. Se regole del modello o comportamento applicativo vengono incorporati direttamente in una superficie, ogni adapter può diventare una variante autonoma del tool e una fonte concorrente di semantica.

Occorre quindi scegliere dove risiedono le capacità ThreatForge rispetto alle superfici che le invocano.

Decision ThreatForge separa le **capacità riutilizzabili del tool**, l'orchestrazione applicativa e gli adapter di delivery/authoring. Le capacità consumano i contratti governati del modello senza ridefinirli; l'applicazione orchestra i casi d'uso; gli adapter traducono interazione e trasporto senza duplicare regole canoniche o logica applicativa.

VS Code costituisce una prima superficie di authoring, non il proprietario delle regole che espone.

Consequences

- Benefit: CLI, editor, CI e applicazione possono riusare le stesse capacità.
- Benefit: il cambio di adapter non richiede una nuova interpretazione del modello DDTA.
- Cost: i confini tra semantica consumata, comportamento applicativo e delivery devono essere mantenuti esplicitamente.
- Risk: logica del modello o dell'applicazione può rifluire negli adapter per convenienza locale.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge product architecture.**
- La semantica dei documenti governati resta esterna al tool; questa ADR governa soltanto la separazione delle responsabilità interne di ThreatForge.

MR-0002 / ADR-0002 - Operazione governata di commit e push nella root canonica

Historical ADR semantic body

Context La promozione della root canonica ha escluso correttamente i tool operativi del progetto legacy, ma ha lasciato ThreatForge senza un endpoint governato per verificare, creare un commit e pubblicarlo.

L'uso diretto di `git commit` e `git push` renderebbe facoltativi i gate locali e ricreerebbe una procedura manuale non verificabile.

Il runner legacy rimane un riferimento storico e non rientra nella root operativa.

Le proiezioni deterministiche versionate nel repository, come schemi e adapter generati dalle fonti canoniche, possono diventare obsolete quando cambiano registri o tassonomie. Comandi `--write` separati renderebbero il contenuto del commit dipendente da una procedura manuale esterna al runner governato.

Decision ThreatForge introduce nella root canonica un nuovo runner CLI sottile per le operazioni di repository.

Il runner:

- offre una modalità `--check` priva di mutazioni Git e scritture nel repository;
- offre una modalità `--commit-push` con messaggio obbligatorio;
- verifica repository, branch e upstream configurato;
- materializza in modalità `--commit-push` le proiezioni deterministiche attive dichiarate da un registro canonico prima del `repo-check`;
- verifica i confini di scrittura, la validità e l'idempotenza di ogni proiezione materializzata;
- mantiene `tools/repo-check.mjs` come gate esclusivamente read-only;
- esegue il `repo-check` canonico prima di qualsiasi stage, commit o push;
- esegue `git add --all` soltanto dopo il superamento della materializzazione e di tutti i gate;
- controlla il diff staged prima del commit;
- interrompe l'operazione al primo errore;
- ripristina i soli output materializzati quando la fase pre-stage fallisce;
- usa il normale upstream Git senza incorporare nomi di remote o branch.

Il runner non contiene la logica dei singoli checker o materializzatori. Il registro letto da `tools/repo-check.mjs` rimane la fonte canonica dell'insieme dei gate; un registro separato letto dal runner rimane la fonte canonica dei materializzatori e dei relativi output dichiarati.

Consequences

- Benefit: Il comando governato è disponibile senza dipendere dal progetto legacy.
- Benefit: CLI, VS Code, app ThreatForge e CI invocano lo stesso entrypoint.
- Benefit: Le proiezioni deterministiche vengono aggiornate e incluse nello stesso commit delle fonti canoniche che le modificano.
- Benefit: L'autore non esegue manualmente comandi `--write` nel normale flusso di commit e push.
- Constraint: `repo-check` e la modalità `--check` del runner restano privi di side effect.
- Constraint: Ogni modifica al runner resta collegata a Requirement e controlli deterministici della root canonica.

Revised candidate ADR

Revised title Pubblicazione governata del repository ThreatForge

Context Nel repository di sviluppo ThreatForge, commit e push diretti possono pubblicare uno stato in cui proiezioni versionate sono obsolete o i gate locali non sono stati eseguiti. Affidare la sequenza corretta alla memoria dello sviluppatore rende la governance del repository facoltativa.

Occorre quindi decidere se la pubblicazione di **ThreatForge stesso** resti una sequenza manuale di comandi Git oppure un'operazione governata del progetto.

Decision Il progetto ThreatForge usa una **operazione governata di pubblicazione Git** che esegue le precondizioni dichiarate dal repository prima di stage, commit e push e mantiene distinta una modalità di sola verifica priva di mutazioni Git.

L'operazione orchestra gate e materializzazioni posseduti dalle rispettive fonti senza duplicarne la logica.

Consequences

- Benefit: la normale pubblicazione di ThreatForge incorpora le verifiche del progetto invece di affidarsi alla disciplina individuale.
- Benefit: fonti e proiezioni versionate possono essere pubblicate in uno stato coerente.

- Cost: il workflow del repository è più strutturato del Git diretto.
- Risk: un orchestratore errato può introdurre side effect o staging inattesi.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge project operations.** Questa ADR non implica che ogni progetto DDTA debba usare Git, lo stesso runner o lo stesso workflow di pubblicazione.
- Sequenza esatta, rollback, upstream e staging sono downstream implementation/operational requirements.

MR-0002 / ADR-0003 - Materializzazione governata di scaffold implementativi

Historical ADR semantic body

Context La pianificazione guidata introdotta da ADR-0001 valida Requirement, tipo, titolo, path, tracciabilità e comando di verifica senza modificare il repository.

Il passo successivo abilita l'avvio del lavoro tecnico da VS Code senza creare codice scollegato, sovrascrivere file esistenti o dichiarare completa un'implementazione ancora vuota.

Decision ThreatForge introduce la materializzazione governata di scaffold implementativi.

La capacità:

- riusa il contratto di validazione del pianificatore esistente;
- crea un solo file sorgente iniziale con tracciabilità obbligatoria;
- registra lo stesso artefatto nell'implementation trace registry;
- usa lo stato canonico **scaffolded** finché l'implementazione non è completata;
- rifiuta path già esistenti o già registrati;
- applica file e record di registro come una singola operazione con rollback in caso di errore;
- è richiamabile da VS Code tramite un adapter sottile.

Lo scaffold non costituisce implementazione completata. Una successiva operazione governata gestisce il passaggio da **scaffolded** a **implemented**.

Consequences

- Benefit: Lo sviluppatore inizia dal Requirement selezionato senza scrivere manualmente gli header di tracciabilità.
- Benefit: Il repository evita scaffold non registrati e record privi del relativo file.
- Benefit: L'implementation trace distingue pianificazione, scaffold e implementazione completata.
- Benefit: VS Code continua a invocare capacità CLI indipendenti dall'IDE.
- Constraint: L'apertura automatica del file e una futura estensione VS Code restano incrementi separati.

Revised candidate ADR

Revised title Supporto ThreatForge all'avvio governato della realizzazione

Context Quando ThreatForge aiuta a passare da un obbligo governato al lavoro tecnico, la creazione manuale di un file può perdere il collegamento con il contesto che ne giustifica l'esistenza o essere scambiata per realizzazione completata.

Occorre quindi decidere come il tool supporti l'avvio della realizzazione senza inventare autonomamente la semantica degli stati del modello.

Decision ThreatForge supporta la **materializzazione di uno scaffold** come operazione esplicita collegata al contesto governato selezionato e alla tracciabilità prevista dal modello. Lo scaffold crea un punto di partenza tecnico ma non costituisce, da solo, evidenza che l'obbligo governato sia soddisfatto.

Gli eventuali stati formali della realizzazione e le condizioni con cui cambiano sono consumati dal modello/progetto e non definiti implicitamente dalla presenza del file.

Consequences

- Benefit: l'avvio del lavoro conserva il legame con il contesto governato.
- Benefit: la presenza di uno scaffold non viene confusa con completamento.
- Cost: la creazione tecnica richiede un'operazione tool-aware aggiuntiva.
- Risk: hard-code di stati o transizioni nel tool può anticipare un lifecycle non ancora definito dal modello.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge product capability.**
- Il termine di classe `Requirement` e gli stati `scaffolded/implemented` non sono necessari per definire questa scelta e restano downstream/deferred.

MR-0002 / ADR-0004 - Authoring guidato della documentazione governata

Historical ADR semantic body

Context I controlli deterministici intercettano divergenze dopo la scrittura della documentazione. Questa protezione è necessaria, ma non elimina il rischio operativo di duplicare titoli, identificativi, path e campi controllati in più file.

La documentazione governata richiede coerenza tra registri YAML, body Markdown, identificativi, titoli, path e valori controllati. La scrittura manuale di questi elementi aumenta la probabilità di errore a ogni nuovo Requirement o Decision.

Il modello distingue tre livelli:

- guida editoriale durante la scrittura;
- generatori deterministici che creano record e body coerenti;
- check finali che impediscono divergenze residue.

Decision ThreatForge supporta l'autoring guidato della documentazione governata tramite strumenti locali e deterministici.

Il registro rimane la fonte canonica strutturata per campi come `id`, `title`, `status`, `body_path` e relazioni. Il body Markdown ripete i dati necessari alla lettura senza diventare fonte autonoma dei campi strutturati.

I generatori creano insieme il record di registro e il body associato, derivando automaticamente identificativi, path e header Markdown quando possibile.

L'autoring guidato dei Requirement deriva dalle fonti canoniche un catalogo deterministico di Macro-requirement, Decision, Requirement padre, valori controllati, significati e regole applicabili. Il medesimo catalogo alimenta lo schema editoriale e il wizard CLI senza duplicare enum o descrizioni nel codice.

Il flusso separa preview e creazione confermata. La creazione applica atomicamente record e body e poi esegue i controlli governati applicabili. **specialized** non è un tipo concreto né un alias di **governance**.

Il core di authoring rimane indipendente dall'IDE e riutilizzabile da CLI, editor, app e automazioni.

I check restano la rete di sicurezza finale senza costituire l'unico meccanismo di coerenza.

Consequences

- Benefit: Gli autori evitano la creazione manuale di identificativi e path derivabili.
- Benefit: VS Code ed editor equivalenti possono usare JSON Schema, task e snippet per guidare la compilazione.
- Benefit: I generatori CLI costituiscono il primo livello implementativo prima di estensioni editor dedicate.
- Constraint: Ogni generatore resta tracciato come artefatto implementativo e collegato ai Requirement supportati.

Non-goals

- Introdurre immediatamente una estensione VS Code dedicata
- Sostituire i controlli deterministici esistenti
- Rendere il body Markdown fonte canonica dei campi strutturati

Revised candidate ADR

Revised title Authoring ThreatForge guidato dai contratti governati del modello

Context La sola validazione a posteriori costringe autori e LLM a ricostruire manualmente identità, relazioni, struttura e valori controllati, aumentando la probabilità di creare documenti incoerenti prima che i check li rilevino.

Occorre quindi decidere se ThreatForge limiti il proprio supporto ai controlli finali oppure usi la conoscenza governata del modello anche durante l'autoring.

Decision ThreatForge offre **authoring guidato e deterministico** derivando suggerimenti, generatori e vincoli dai contratti governati del modello/progetto anziché mantenere copie locali delle regole nei singoli adapter.

L'autoring riduce gli errori prima della validazione, mentre i check restano un controllo indipendente finale. Il tool guida l'uso del modello ma non diventa la fonte della sua semantica.

Consequences

- Benefit: persone e LLM ricevono il contesto rilevante durante la scrittura invece di dover ricordare tutte le Decision precedenti.
- Benefit: CLI/editor possono condividere la stessa conoscenza derivata.
- Cost: proiezioni e contratti di authoring devono restare freschi e tracciabili alla fonte governata.
- Risk: assistenza obsoleta può guidare verso contenuto incompatibile con il modello corrente.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge product capability.**
- Questa ADR è un possibile consumer futuro dell'**effective governed context**, ma non ne definisce ancora il meccanismo generale.
- Atomicità, preview, wizard e dettagli di generazione sono downstream.

MR-0002 / ADR-0005 - Integrazione locale di VS Code tramite task governati

Historical ADR semantic body

Context I tool locali governati sono disponibili come comandi CLI, ma il loro uso quotidiano richiede di ricordare path, argomenti e modalità sicure di esecuzione.

L'integrazione editor riduce questo attrito senza duplicare logica, introdurre prematuramente una custom extension o trasformare VS Code in una nuova fonte canonica delle regole operative.

I task coprono check locali, authoring guidato e produzione dell'handoff. Per questo motivo formano un livello trasversale e non una semplice estensione del solo generatore documentale governato da MR-0002/ADR-0004 o della Decision sull'handoff MR-0001/ADR-0006.

Decision ThreatForge adotta un catalogo locale di task VS Code come livello di lancio sottile sopra i comandi CLI governati esistenti.

Il catalogo:

- usa label visibili con prefisso canonico **ThreatForge::**;
- invoca direttamente i tool esistenti senza replicarne la logica;
- esegue i comandi dalla root del repository;

- mantiene separati i task di preview dai task che producono side effect;
- consente la creazione di documenti governati delegando al core CLI la preview e la conferma esplicita con valore predefinito non distruttivo;
- mantiene in `--dry-run` la produzione dell'handoff fino a un Requirement successivo sulla creazione confermata;
- esclude da task, input, setting, associazioni di schema e snippet ogni copia delle regole e dei valori canonici;
- risiede sotto `.vscode/tasks.json` nella root canonica del repository;
- non introduce una custom extension VS Code.

I task vengono caricati aprendo la root del repository ThreatForge come cartella workspace in VS Code. Il `cwd` coincide con `${workspaceFolder}`.

Consequences

- Benefit: I check e i generatori locali diventano accessibili dalla palette **Tasks: Run Task**.
- Benefit: Le label utente espongono esclusivamente il nome canonico **ThreatForge**.
- Benefit: Le modifiche ai comandi restano governate nei tool CLI anziché nel catalogo editor.
- Constraint: Schemi, snippet e setting dedicati entrano come incrementi separati.
- Constraint: Una custom extension resta fuori ambito finché i task locali risultano sufficienti.

Revised candidate ADR

Revised title VS Code task come adapter locale sottile di ThreatForge

Context Le capacità ThreatForge disponibili via CLI sono riutilizzabili ma poco ergonomiche durante il lavoro quotidiano in VS Code. Duplicare regole o valori nell'editor creerebbe una seconda implementazione, mentre una integrazione più complessa non è necessaria per semplici operazioni di lancio.

Occorre quindi scegliere la prima superficie locale con cui esporre tali capacità nell'editor.

Decision ThreatForge usa un **catalogo di task VS Code** come adapter locale sottile sopra gli endpoint governati esistenti. I task espongono operazioni e input senza possedere semantica del modello, valori canonici o logica applicativa e mantengono distinguibili operazioni read-only e operazioni con side effect.

Consequences

- Benefit: le operazioni frequenti diventano accessibili dall'editor senza una seconda implementazione delle regole.
- Benefit: la CLI/capacità sottostante resta riutilizzabile fuori da VS Code.
- Cost: il catalogo deve seguire l'evoluzione degli endpoint.
- Risk: aggiungere regole locali ai task può trasformare progressivamente l'adapter in una fonte concorrente.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge adapter decision.** Non è una regola DDTA né una prescrizione per gli editor dei target project.
- Path, label, `cwd`, `dry-run` e input specifici sono downstream.

MR-0002 / ADR-0006 - Shared Markdown assistance core and thin editor adapters

Historical ADR semantic body

Context The governed Requirement authoring request and VS Code tasks provide safe creation workflows, but they cannot inspect the current unsaved Markdown body, determine the next missing section or emit live body diagnostics at the cursor position.

MR-0002/ADR-0005 intentionally excluded a custom VS Code extension while tasks were sufficient. Context-sensitive body completion, hover, diagnostics and quick fixes show that tasks are no longer sufficient for this authoring surface. The existing task catalog remains valid for governed commands and repository mutations.

The same assistance is intended for the Governance Console web editor. Separate canonical rule implementations in VS Code and the web frontend would create competing sources and divergent diagnostics.

Decision ThreatForge adopts an editor-independent Markdown assistance core. The core receives document identity, repository-relative path, current unsaved text, cursor position and canonical document model projections. It returns completions, diagnostics, hover information and applicable quick fixes without modifying the document or repository.

The core identifies the applicable governed body profile, proposes the next missing required section, proposes controlled labels only in applicable controlled sections and reports missing, duplicate, unknown or out-of-order sections. It also reports invalid normative forms, invalid punctuation and divergence between mirrored registry and body values.

A dedicated VS Code extension or provider acts as a thin adapter over the shared core. A future Governance Console editor adapter consumes the same analysis contract. Editor adapters contain no canonical section inventories, value sets or validation rules.

The initial implementation uses the shared core directly without a Language Server. A future Language Server can wrap the same core while preserving the canonical analysis semantics.

Consequences

- Benefit: Typing **##** can propose the next valid canonical section for the current body.
- Benefit: VS Code and the future web editor can emit equivalent diagnostics for identical document state.
- Cost: ThreatForge packages and maintains a dedicated thin VS Code integration.
- Risk: Adapter-specific transformations could cause divergent ranges or completion ordering.
- Constraint: The analysis core remains side-effect-free and consumes canonical model projections.

Non-goals

- Automatic silent normalization of invalid controlled values
- Direct mutation of canonical registries from live body analysis
- Initial support for editors other than VS Code and the Governance Console

Revised candidate ADR

Revised title Core ThreatForge condiviso per assistenza Markdown contestuale

Context I task possono lanciare operazioni ma non interpretano il testo non salvato e la posizione del cursore. Se VS Code e un editor web implementassero separatamente completion e diagnostica, ciascun adapter potrebbe ricostruire una propria versione delle regole documentali.

Occorre quindi decidere dove ThreatForge esegua l'analisi contestuale usata dall'assistenza live.

Decision ThreatForge usa un **core di assistenza Markdown indipendente dall'editor e privo di side effect** che consuma i contratti/proiezioni governate del modello e restituisce informazione di assistenza sul testo corrente. VS Code e altri editor restano adapter di presentazione e non possiedono inventari o regole semantiche autonome.

Consequences

- Benefit: editor differenti possono produrre assistenza semanticamente coerente sullo stesso stato documentale.
- Benefit: il core può essere verificato indipendentemente dall'interfaccia.
- Cost: il contratto tra core e adapter deve restare stabile.
- Risk: trasformazioni adapter-specifiche possono ancora produrre differenze di range, ordine o presentazione.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge product architecture.**
- Completion, hover, quick fix, regole di sezione e scelta Language Server sono downstream.
- Il core consuma il modello; non ne è il proprietario semantico.

MR-0002 / ADR-0007 - Architettura applicativa riutilizzabile per adapter CLI e web

Historical ADR semantic body

Context ThreatForge is introducing product capabilities that initially use a command-line interface and later become available through a web application. The legacy ThreatForge implementation already demonstrated a reusable backend shape based on factories, a feature-local composition root, controllers, services, ports, adapters, Zod contracts and transport-independent route descriptors. The current governed repository preserves the separation between core capabilities, the ThreatForge application and delivery adapters, but it does not yet define the complete reusable module shape for new product features. Implementing target-project creation directly inside a CLI runner would couple application behavior to terminal parsing, filesystem access and interactive confirmation, making later web reuse more difficult.

Decision ThreatForge adopts a reusable application architecture for product features exposed through CLI and web delivery adapters. Feature behavior resides in application services that receive validated commands and depend on explicit ports rather than concrete filesystem, Git, database or transport implementations. Adapters implement those ports, while a feature-local factory or composition root creates concrete dependencies and returns the composed feature module. CLI runners remain thin delivery adapters responsible for argument parsing, user interaction, result formatting and process exit behavior. HTTP controllers and route descriptors remain thin delivery boundaries that translate validated transport input into application-service commands and return transport-safe results. Zod represents runtime contracts at application and transport boundaries, and OpenAPI represents the HTTP contract consumed by the future React frontend. React components consume API clients or frontend controller and hook boundaries without reading governed YAML, Markdown, graph, Git or filesystem sources directly. Cross-cutting HTTP behavior remains in middleware, while feature-specific behavior remains in the owning application service. The architecture applies first to target-project creation and later to other reusable ThreatForge product capabilities.

Consequences

- Benefit: CLI and web adapters reuse the same application behavior.
- Benefit: Application services remain independently testable without terminal or HTTP coupling.
- Benefit: Concrete filesystem and future storage implementations remain replaceable behind ports.
- Benefit: Composition roots make dependency ownership and feature assembly explicit.
- Benefit: Zod and OpenAPI provide distinct runtime and HTTP contract boundaries.
- Benefit: React remains isolated from governed repository source formats.
- Cost: Each product feature requires explicit contracts, services, ports, adapters and composition.
- Cost: Initial implementation requires more files than a single command-line runner.
- Cost: CLI and HTTP adapters require separate verification against shared service behavior.
- Risk: Excessive abstraction could slow the first demonstrable product.
- Risk: Application logic could leak back into delivery adapters if boundaries are not verified.
- Risk: Legacy patterns could be copied without adapting them to the simplified current model.
- Constraint: Feature behavior belongs to the owning Macro-requirement while reusable architecture belongs to MR-0002.
- Constraint: Controllers and delivery adapters do not instantiate concrete infrastructure adapters.
- Constraint: Services access infrastructure only through explicit ports.
- Constraint: A feature-local composition root owns concrete dependency assembly.
- Constraint: The initial CLI and future web API use the same application-service contract.
- Constraint: The frontend does not read project-model files or filesystem paths directly.
- Constraint: The first implementation remains limited to the architecture needed by target-project creation.

Non-goals

- Define the final web interface
- Select a final HTTP framework
- Define authentication, role or permission semantics
- Define every future storage adapter
- Rebuild the complete legacy application architecture
- Implement target-project creation in this Decision

Revised candidate ADR

Revised title Architettura applicativa ThreatForge indipendente dai delivery adapter

Context Le capacita di prodotto ThreatForge devono poter essere esposte da CLI e successivamente da interfacce web. Incorporare comportamento applicativo, accesso infrastrutturale e traduzione del transport nello stesso runner renderebbe ogni nuova superficie una migrazione o una duplicazione.

Occorre quindi scegliere un confine applicativo riutilizzabile interno al tool.

Decision ThreatForge organizza le capacita di prodotto riutilizzabili attorno a **servizi applicativi con dipendenze esplicite verso porte**, implementate da adapter infrastrutturali e composte in un confine di assembly della feature. CLI, HTTP e altre superfici traducono input/output ma non possiedono il comportamento applicativo.

Questa architettura organizza il tool ThreatForge; non definisce la struttura interna dei progetti governati ne la semantica del metamodello DDTA.

Consequences

- Benefit: CLI e web possono condividere comportamento e test applicativi.
- Benefit: infrastruttura e transport restano sostituibili dietro confini espliciti.
- Cost: una feature richiede responsabilita e assembly piu espliciti di un runner monolitico.
- Risk: astrazioni premature o leakage di logica negli adapter possono annullare il vantaggio.

Redistribution / ownership notes

- **Ownership candidate: ThreatForge product architecture.**
- Zod, OpenAPI, React, framework HTTP e vincoli della prima feature sono downstream o candidate Decision separate se acquistano trade-off autonomi.

Cross-case observations after ownership-aware rewrites

Revision 4 changes the interpretation of several historical ADRs without changing their historical text.

Ownership patterns exposed by the construction corpus

Historical topic	Revised semantic owner candidate
Diátaxis	DDTA method
Controlled vocabulary	DDTA method / cross-document governance
Implementation trace	split: DDTA semantics + ThreatForge support
Controlled labels/value sets	ThreatForge support; semantics external
Handoff	ThreatForge project operations
Canonical document models	ThreatForge model-consumption architecture
Governed reference grammar	DDTA Doc-as-Code representation
Security Requirement specialization	split: future metamodel + ThreatForge support
Model extensibility	ThreatForge consumer architecture
Core/app/editor separation	ThreatForge product architecture
Commit/push	ThreatForge project operations
Scaffold	ThreatForge product capability
Guided authoring	ThreatForge product capability
VS Code tasks	ThreatForge adapter
Markdown assistance core	ThreatForge product architecture
CLI/web application architecture	ThreatForge product architecture

The table is **not a new taxonomy field** for Decision. It is a review result used to detect historical ownership conflation.

Main lesson

A rewrite is not correct merely because it is shorter. It must preserve the actual choice **and place that choice with the correct semantic owner**.

```
metamodel/method choice
    !=
ThreatForge support choice
    !=
ThreatForge development-operation choice
```

A single historical ADR may contain more than one of these and may therefore require later migration or splitting after the full model is closed.

Construction candidate now fixed elsewhere

DDTA_DECISION_METAMODEL_WORKING revision 4 marks the minimal Decision core and semantic-owner separation as closed for the construction candidate. These decisions can be reopened only by an explicit counterexample from the independent example or sequential holdouts.

Still open are the Requirement semantics, cross-document propagation/effective context, lifecycle, serialization closure and the actual migration of ThreatForge historical ADRs.